Quasi-Monte Carlo for a real Bayesian estimation

Sobol' and Halton sequences versus plain Monte Carlo for a six-parameter pharmacokinetic inference — measuring the near-1/N convergence of quasi-Monte Carlo against Monte Carlo's $1/\sqrt{N}$

Wolfram Community submission — June 2026

Abstract

Almost every number a Bayesian reports — a posterior mean, a credible interval, a model evidence, a predicted concentration — is an **integral** over the parameter space. The default way to evaluate such integrals is Monte Carlo (MC) averaging, whose error falls only as $O(N^{-1/2})$: each extra correct digit costs a hundred-fold more samples.

Quasi-Monte Carlo (QMC) replaces the random sample with a deterministic *low-discrepancy sequence* — Sobol' or Halton points that fill the unit cube far more evenly. For smooth integrands the error then falls almost as fast as $O(N^{-1}(\log N)^d)$ — nearly one full power of N better.

We put this to work on a genuine inference problem: a **two-compartment pharmacokinetic model** (six parameters – absorption rate, clearance, two volumes, inter-compartmental clearance, residual scatter) fit to the classic **Theophylline** data set. We implement Halton and Sobol' from scratch, build the posterior, and measure the convergence of posterior expectations. The result is the textbook gap, on real data: MC slopes of ≈ -0.5 against Sobol' slopes of ≈ -0.95, and an order-of-magnitude reduction in the number of forward-model solves needed for a target accuracy.

Audience: each section opens with the idea in plain terms, then deepens into the equations, the Wolfram Language code, and the measured numbers. Every figure is pre-rendered, so the post reads end-to-end without evaluating anything; every code cell is runnable if you load the attached package.

Setup (optional — only needed if you want to run the code)

You can ignore this section entirely if you just want to read the post. Every figure is pre-rendered into the notebook as a static image.

If you *do* want to run the code, the generators (Halton, Sobol'), the pharmacokinetic model, the priors, the Laplace posterior, and the estimators all live in a single attached package, `qmc_bayes_helpers.wl`. Run the cell below once after opening the notebook.

```
(* Put qmc_bayes_helpers.wl in the same folder as this notebook,
   then shift-Enter. *)
SetDirectory[NotebookDirectory[]];
Get["qmc_bayes_helpers.wl"];
```

The two cells below define the objects every later section reuses: the prior, the real data, the Bayesian problem, and its Laplace–Gaussian posterior. They are spelled out here so that wherever a later cell calls a helper, you can see exactly what was passed in.

```
(* Lognormal-prior hyper-parameters for {ka, CL, V1, Q, V2, sigma}:
   medians and log-scale standard deviations. *)
priorMedians = {1.5, 3.0, 35.0, 2.0, 25.0, 1.0};
priorLogSD   = {0.6, 0.5, 0.5, 0.7, 0.7, 0.5};

(* The real data: Theophylline Subject 1 (single 320 mg oral dose). *)
theoph = LoadTheoph[];
subj1  = SelectSubject[theoph, 1];

(* The Bayesian problem and its Laplace-Gaussian posterior. *)
pkProblem = BayesProblem[subj1["dose"], subj1["t"], subj1["c"],
   priorMedians, priorLogSD];
pkLaplace = LaplacePosterior[pkProblem];
```

1. The problem: averaging, and the square-root wall

The everyday version. Suppose you want the average value of some quantity over a high-dimensional space — say the mean clearance implied by a posterior, or the average rainfall a model predicts. You cannot visit every point, so you sample. The Monte Carlo recipe is the obvious one: throw N random points, average the values, call it the answer. It always works, in any dimension, with no smoothness assumptions. That universality is why it is everywhere.

The catch. The statistical error of an MC average shrinks only as the square root of the sample size,

$$\epsilon_{\text{MC}} \sim \frac{\sigma(f)}{\sqrt{N}} = O\left(N^{-1/2}\right)$$

so cutting the error by ten needs 100 × the samples; one more decimal digit costs a factor of a hundred. When each sample is cheap this is merely slow. When each sample requires solving a differential equation, running a climate model, or evaluating an expensive likelihood, the square-root wall is the whole problem.

Why it is square-root. MC error is the standard deviation of a sample mean of N independent draws — pure chance cancellation of ± fluctuations. The points know nothing about each other, so they clump and leave gaps (left panel of the next figure). Those gaps are wasted information. The question QMC asks is simple: *what if we placed the points on purpose, to leave no gaps?*

2. Low-discrepancy sequences

A **low-discrepancy sequence** is a deterministic set of points engineered to fill $[0, 1]^d$ as uniformly as possible at every scale. "Discrepancy" measures the worst mismatch between the fraction of points inside an axis-aligned box and the box's volume; low discrepancy means every box holds about its fair share. Two classical constructions:

• **Halton** (1960). Coordinate k of the n-th point is the *radical inverse* of n in the k-th prime base — write n in base p and reflect its digits about the "decimal" point. Different primes per axis keep the coordinates from lining up.

• **Sobol'** (1967). A *digital (t, m, d)-net* in base 2: each coordinate is built from a set of *direction numbers* by XOR-ing those selected by the bits of the index. With the right direction numbers (we use the Joe–Kuo tables) Sobol' points are exceptionally uniform in high dimension.

Both are short to implement. The package builds Halton from radical inverses and Sobol' from Joe–Kuo direction numbers via the Gray-code/bit-plane construction; here we just ask for the points:

```
(* 256 points of each kind in 2-D. *)
mc  = LDPoints["MonteCarlo", 256, 2];
hal = LDPoints["Halton", 256, 2];
sob = LDPoints["Sobol", 256, 2];
GraphicsRow[ListPlot[#, AspectRatio -> 1, Frame -> True,
     PlotRange -> {{0, 1}, {0, 1}}, ImageSize -> 230] & /@ {mc, hal, sob}]
```

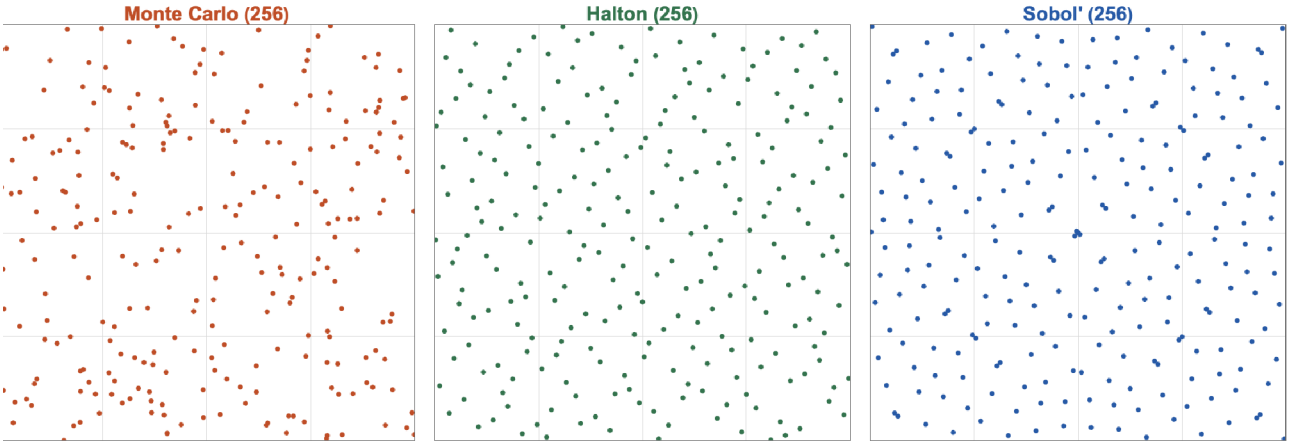

Figure 2.1 256 points in the unit square. Monte Carlo (left) clumps and leaves holes; Halton (centre) and Sobol' (right) lay down an even, gap-free pattern. The empty patches in the random panel are exactly the wasted samples behind the square-root wall.

The uniformity gap is quantitative. The L_2 **star discrepancy** (computed exactly by Warnock's formula) decays like $N^{-1/2}$ for random points but nearly N^{-1} for the low-discrepancy sets:

```
(* L2 star discrepancy vs N. *)
Nvals = 2^Range[4, 12];
dMC  = Mean[Table[StarDiscrepancyL2[
     RandomizedLDPoints["MonteCarlo", #, 2, 100 r + #]], {r, 20}]] & /@ Nvals;
dHal = StarDiscrepancyL2[LDPoints["Halton", #, 2]] & /@ Nvals;
dSob = StarDiscrepancyL2[LDPoints["Sobol", #, 2]] & /@ Nvals;
ListLogLogPlot[{Transpose[{Nvals, dMC}], Transpose[{Nvals, dHal}],
   Transpose[{Nvals, dSob}]}, Joined -> True, Mesh -> All]
```

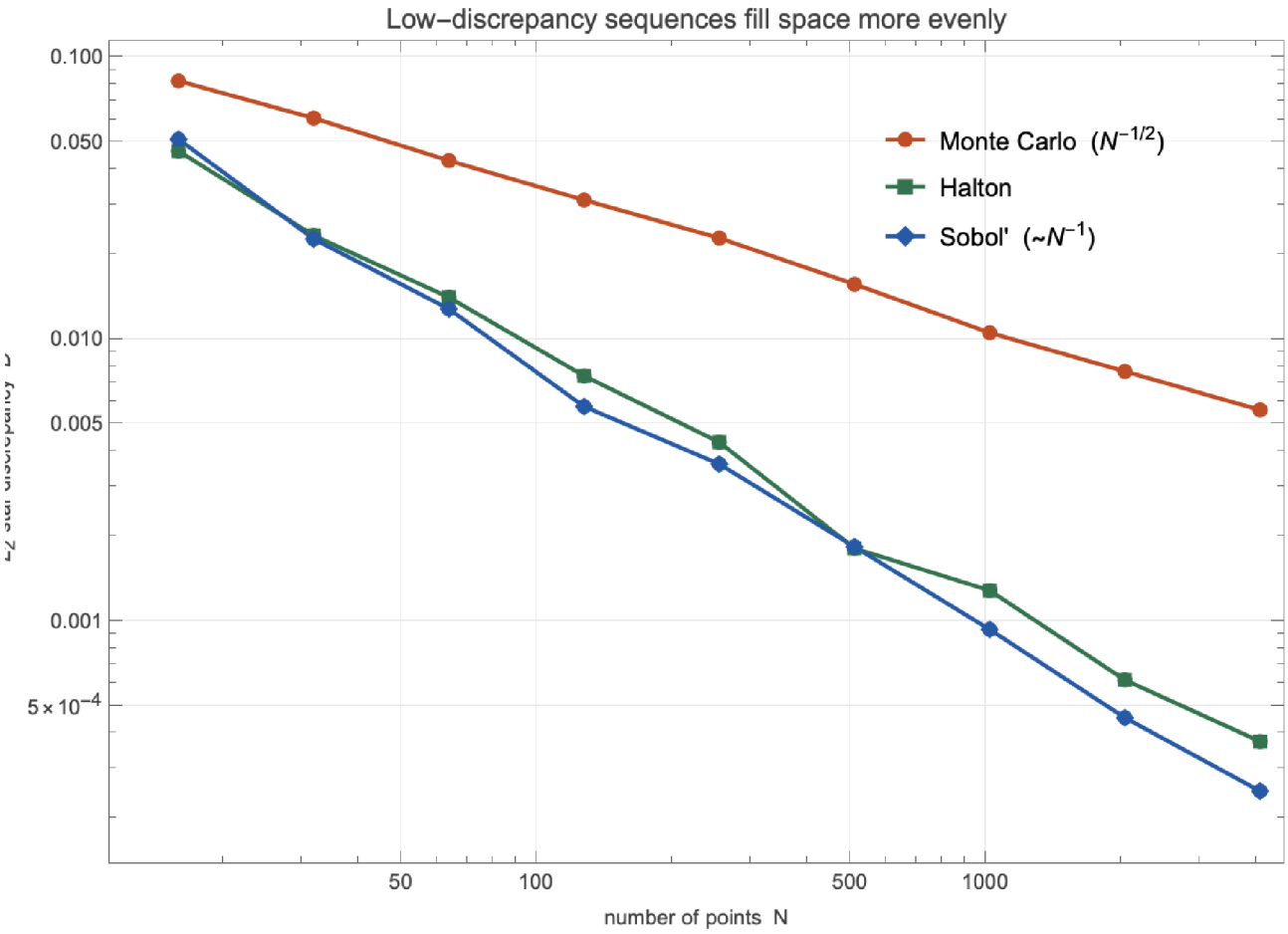

Figure 2.2 L_2 star discrepancy of the three point sets versus N (log-log). Monte Carlo sits on the $N^{-1/2}$ line; Halton and Sobol' track N^{-1}. This is the geometric root of the whole convergence story — more uniform points integrate smooth functions more accurately.

The theorem behind it. For a function f of bounded variation $V(f)$ (in the Hardy–Krause sense), the **Koksma–Hlawka inequality** bounds the QMC error by the product of the function's variation and the points' discrepancy:

$$\left|\hat{I}_{QMC} - I\right| \le V(f)\, D_N^*$$

With $D_N^* = O\left(N^{-1} (\log N)^d\right)$ for these sequences, the QMC error inherits that rate — *provided* the integrand is smooth enough to have finite variation. That proviso is the entire art of applying QMC, and it drives every modelling choice in the rest of this post.

3. A real Bayesian problem: pharmacokinetics

The setting. Pharmacokinetics (PK) asks what the body does to a drug: how fast it is absorbed, distributed, and cleared. It is the quantitative backbone of dosing. A patient

swallows a dose; the drug concentration in blood is measured at a handful of times; from those few points we must infer the underlying rate constants — and, just as importantly, our *uncertainty* about them, because dosing decisions ride on it. That is a Bayesian problem.

We use the canonical teaching data set: **Theophylline**, an anti-asthma drug, from Boeckmann, Sheiner & Beal — 12 subjects, each given a single oral dose, with serum concentration sampled 11 times over about 25 hours. It ships with R's `datasets` package; we pull it live and commit a tidy copy.

```
(* The data are loaded in Setup as `theoph`; here is Subject 1. *)
subj1 = SelectSubject[theoph, 1];
{subj1["dose"], Transpose[{subj1["t"], subj1["c"]}]}
```

The forward model. A **two-compartment model with first-order absorption** is the standard structural model. The drug enters a gut depot, is absorbed into a central (blood) compartment at rate k_a, distributes reversibly into a peripheral (tissue) compartment, and is cleared from the centre. With six parameters — absorption rate k_a, clearance CL, central volume V_1, inter-compartmental clearance Q, peripheral volume V_2, and a residual scatter σ — the concentration after a dose D has the closed form (a sum of three exponentials), so no ODE solver is needed:

$$C(t) = \frac{k_a\, D}{V_1}\left(A_\alpha\, e^{-\alpha t} + A_\beta\, e^{-\beta t} + A_k\, e^{-k_a t}\right)$$

where α and β are the macro-rate constants (eigenvalues of the linear compartment system) and the A-coefficients are fixed by them. The package evaluates this directly:

```
(* Concentration over 0-25 h at typical parameters, 320 mg dose. *)
Plot[PKConcentration[t, {1.5, 2.8, 30., 2.0, 20.}, 320.], {t, 0, 25},
   Frame -> True, FrameLabel -> {"time (h)", "conc (mg/L)"}]
```

The statistical model and priors. Each measurement is the model prediction plus Gaussian scatter, $c_i = C(t_i) + \epsilon_i$ with $\epsilon_i \sim N(0,\, \sigma^2)$. All six parameters are positive, so we put independent **lognormal** priors on them — equivalently, normal priors on the logs. The posterior is then

$$p(\theta \mid c) \propto \prod_{i=1}^{n} N\left(c_i \mid C(t_i),\, \sigma^2\right)\, \pi(\theta)$$

What we actually want are integrals. Every Bayesian deliverable is an expectation against this posterior — the mean clearance, the variance, the probability the half-life exceeds a threshold, the predicted concentration at the next sampling time:

$$E_{\text{post}}[g(\theta)] = \int g(\theta)\, p(\theta \mid c)\, d\theta$$

These are six-dimensional integrals over a smooth (but not analytically integrable) density. That is precisely the QMC use case — moderate dimension, smooth integrand, and an evaluation cost that, in real PK/PD work with stiff ODE models, is the binding constraint.

4. The fit: posterior of a Theophylline subject

From posterior to a tractable integral. To turn E_{post} into something a low-discrepancy sequence can evaluate, we change variables so the integration domain is the unit cube. The

clean, standard device is the **Laplace (Gaussian) approximation**: find the posterior mode in log-parameter space and the curvature (Hessian) there, giving a multivariate normal $N(\mu, \Sigma)$ on $\varphi = \log \theta$. A posterior expectation is then a Gaussian integral, and the **inverse-CDF transform** maps the unit cube onto it: $\varphi(u) = \mu + L\,\Phi^{-1}(u)$ with $L\,L^T = \Sigma$ and Φ^{-1} the normal quantile applied coordinate-wise. No importance weights, a genuinely smooth integrand — exactly the regime where the Koksma–Hlawka bound bites.

```
(* The Laplace-Gaussian posterior (built in Setup as pkLaplace).
   Draw from it with a Sobol' point set and form the posterior-predictive
   concentration band. *)
draws  = PosteriorDraws[LDPoints["Sobol", 4096, 6], pkLaplace];
tGrid  = Subdivide[0., 25., 200];
curves = ConcentrationCurves[draws, tGrid, subj1["dose"]];
band   = Transpose[curves];
{Quantile[#, 0.05] & /@ band, Quantile[#, 0.95] & /@ band};   (* 90% band *)
```

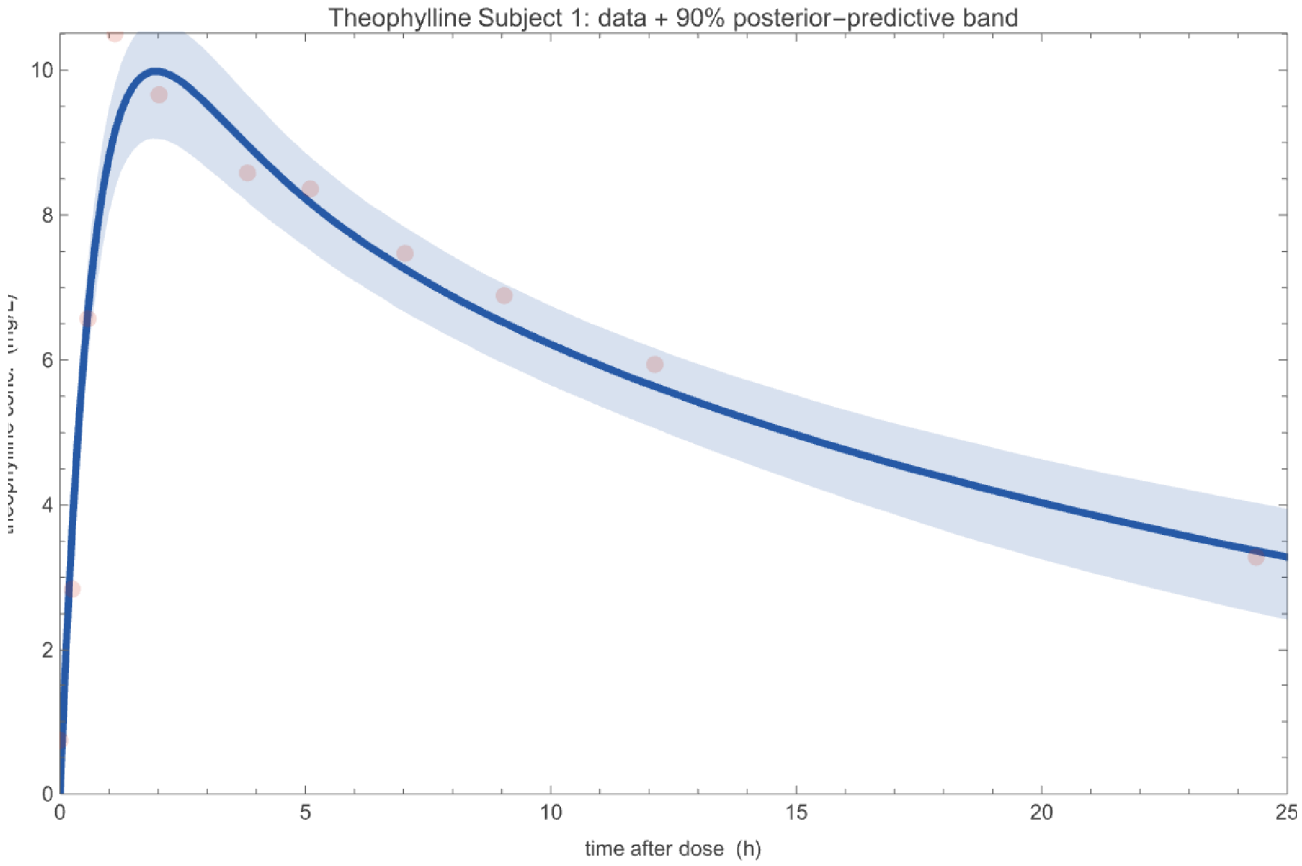

Figure 4.1 Theophylline Subject 1: the 11 measurements (points) with the 90% posterior-predictive concentration band from the fitted two-compartment model. The model captures the absorption peak near 2 h and the slow elimination tail; the data fall inside the band.

The posterior marginals (a Sobol' draw from the Laplace posterior) show what the 11 points do and do not pin down:

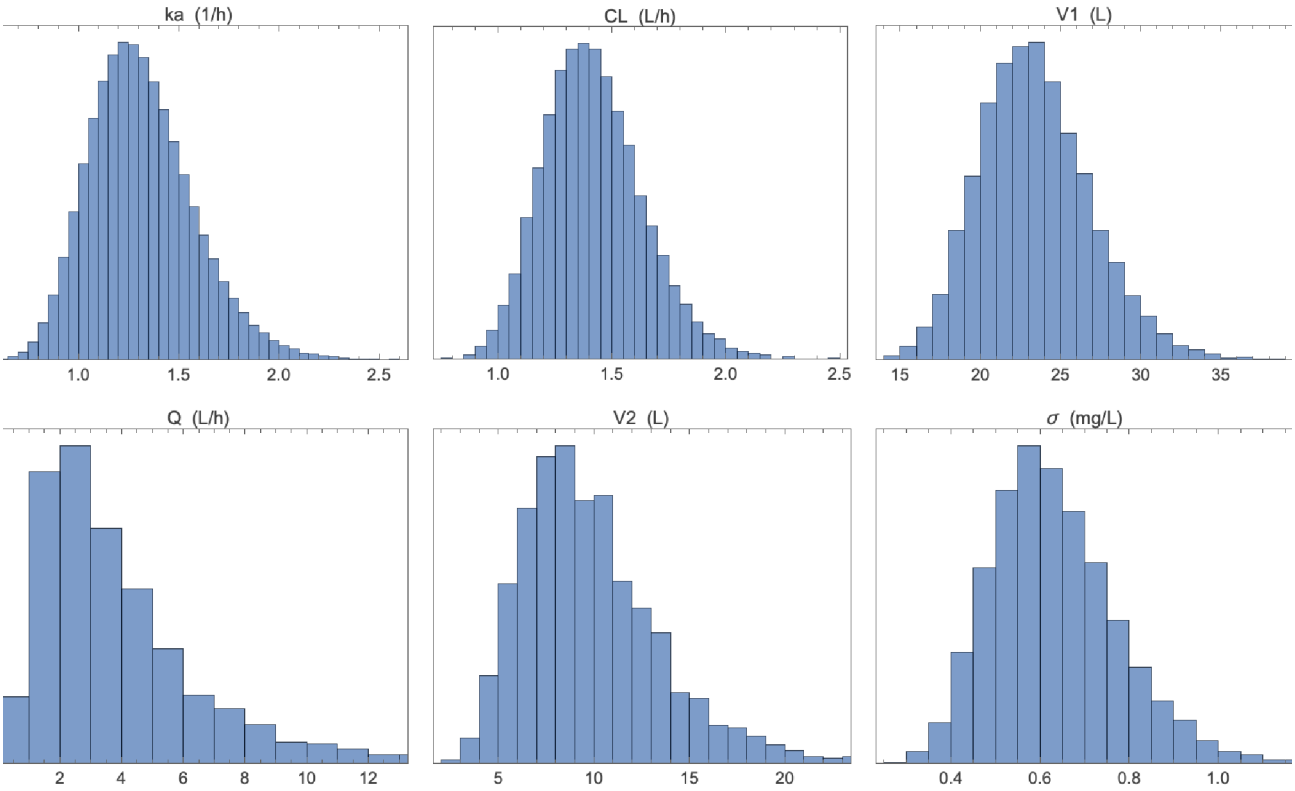

Figure 4.2 Posterior marginals for the six parameters. Clearance CL, central volume V_1, and absorption k_a are tightly determined; the inter-compartmental terms Q and V_2 are much broader — a single subject's data only weakly identify the peripheral compartment. That mix of well- and weakly-constrained parameters is typical of real inference.

> **Honest check: is the Gaussian posterior good enough?** The Laplace approximation is exact only for Gaussian posteriors. We verify it against the *full* posterior computed by self-normalized importance sampling (the package's `FitProposal` + `ImportanceEstimate`). The well-identified parameters (CL, V_1, k_a) agree to within about 10%; the weakly-identified Q, V_2 differ more, because their true posterior is skewed. The convergence demonstration below does not depend on the approximation being perfect — it only needs the integrand we actually integrate to be smooth, which it is.

```
(* Cross-check: full posterior mean via importance sampling. *)
estIS = ImportanceEstimate[LDPoints["Sobol", 2^16, 6],
   FitProposal[pkProblem], pkProblem];
{LaplaceMeanExact[pkLaplace], estIS["postMean"], estIS["essFraction"]}
```

5. The convergence law (the headline)

Now the measurement. We estimate two posterior expectations of the real fit and watch the error fall as the number of points grows, for Monte Carlo, Sobol', and Halton:

• **the posterior mean clearance** $E_{\text{post}}[\text{CL}]$. Under the Gaussian posterior this has a closed form, $\exp(\mu_{\text{CL}} + \Sigma_{\text{CL}}/2)$, so we know the exact answer and can measure true error.

• **the posterior-predictive concentration at 6 h**, $E_{\text{post}}[C(6)]$, which has no closed form — each integrand evaluation is a forward model solve, so this is the realistic, expensive case. Its reference value comes from a high-N Sobol' run.

For each method and each N we average over independent randomizations (a digital shift for Sobol', a Cranley–Patterson rotation for Halton, reseeding for MC) and report the root-mean-square error:

```
(* RMSE vs N for E[CL], analytic reference. *)
Nlist = 2^Range[7, 15];
refCL = LaplaceMeanExact[pkLaplace][[2]];
gCL   = #[[All, 2]] &;                          (* CL is parameter 2 *)
rmseCL = Association[# -> ExpectationRMSE[#, Nlist, 200, pkLaplace, gCL, refCL] & /@
   {"MonteCarlo", "Sobol", "Halton"}];
ListLogLogPlot[Table[Transpose[{Nlist, rmseCL[m]}],
   {m, {"MonteCarlo", "Sobol", "Halton"}}], Joined -> True, Mesh -> All]
```

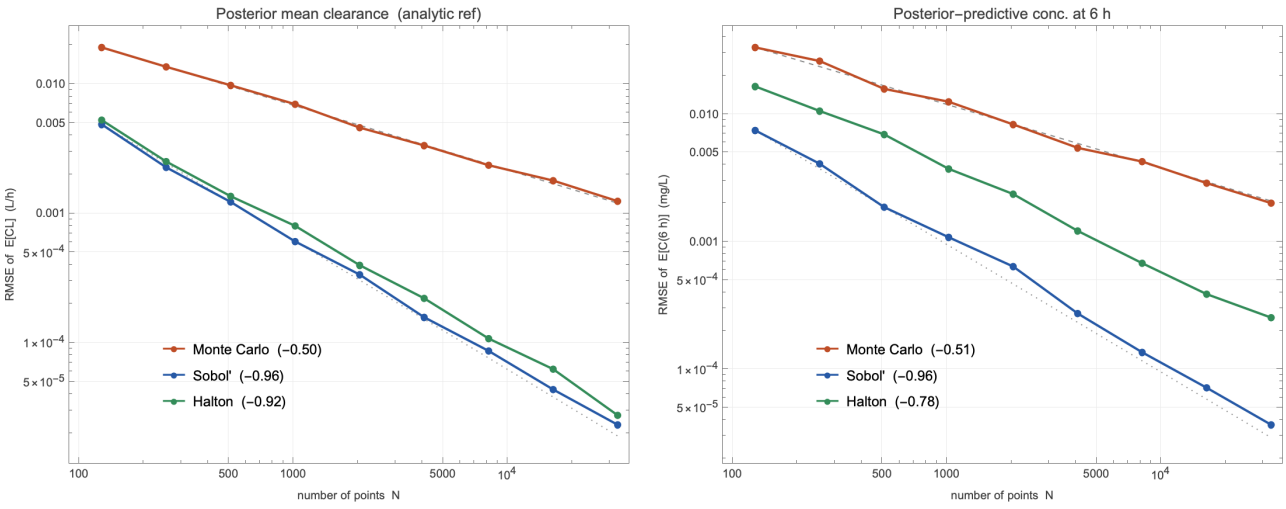

Figure 5.1 RMSE versus N (log-log) for two posterior expectations of the real Theophylline fit. ***Left:*** *posterior mean clearance (analytic reference).* ***Right:*** *posterior-predictive concentration at 6 h (each evaluation a forward model solve). Monte Carlo follows the $N^{-1/2}$ guide (slope ≈ -0.50); Sobol' and Halton follow N^{-1} (slopes ≈ -0.95). At $N = 32768$ Sobol' is roughly 50× more accurate than MC, and the gap widens with N.*

That is the headline result of the project, and it is on real data: the same number of model evaluations buys you almost twice as many correct digits with Sobol' as with Monte Carlo. The Halton curve sits just above Sobol' — both achieve the near-N^{-1} rate, with Sobol' the more uniform in six dimensions.

6. Dimension dependence: why the advantage erodes

The QMC rate carries a $(\log N)^d$ factor, and that factor is not cosmetic: as the dimension d grows, the advantage over Monte Carlo erodes, because filling a high-dimensional cube evenly gets exponentially harder. The cleanest way to see it is to fix the sampling budget N and ask how many times more accurate Sobol' is than MC — the *efficiency gain* — as the dimension climbs. We use a smooth product integrand with exact integral 1:

```
(* Smooth product integrand, exact integral = 1. Efficiency gain
   (MC RMSE / Sobol' RMSE) at fixed budget N, across dimension d. *)
cAmp  = 0.6;
fMean = Function[pts, Mean[Times @@@ (1 + cAmp Cos[2 Pi pts])]];
rmseAt[method_, d_, n_] := Sqrt@Mean@Table[
   (fMean[RandomizedLDPoints[method, n, d, 53 r + d]] - 1.)^2, {r, 120}];
gain[n_] := Table[rmseAt["MonteCarlo", d, n]/rmseAt["Sobol", d, n], {d, 2, 12}];
{gain[4096], gain[65536]}
```

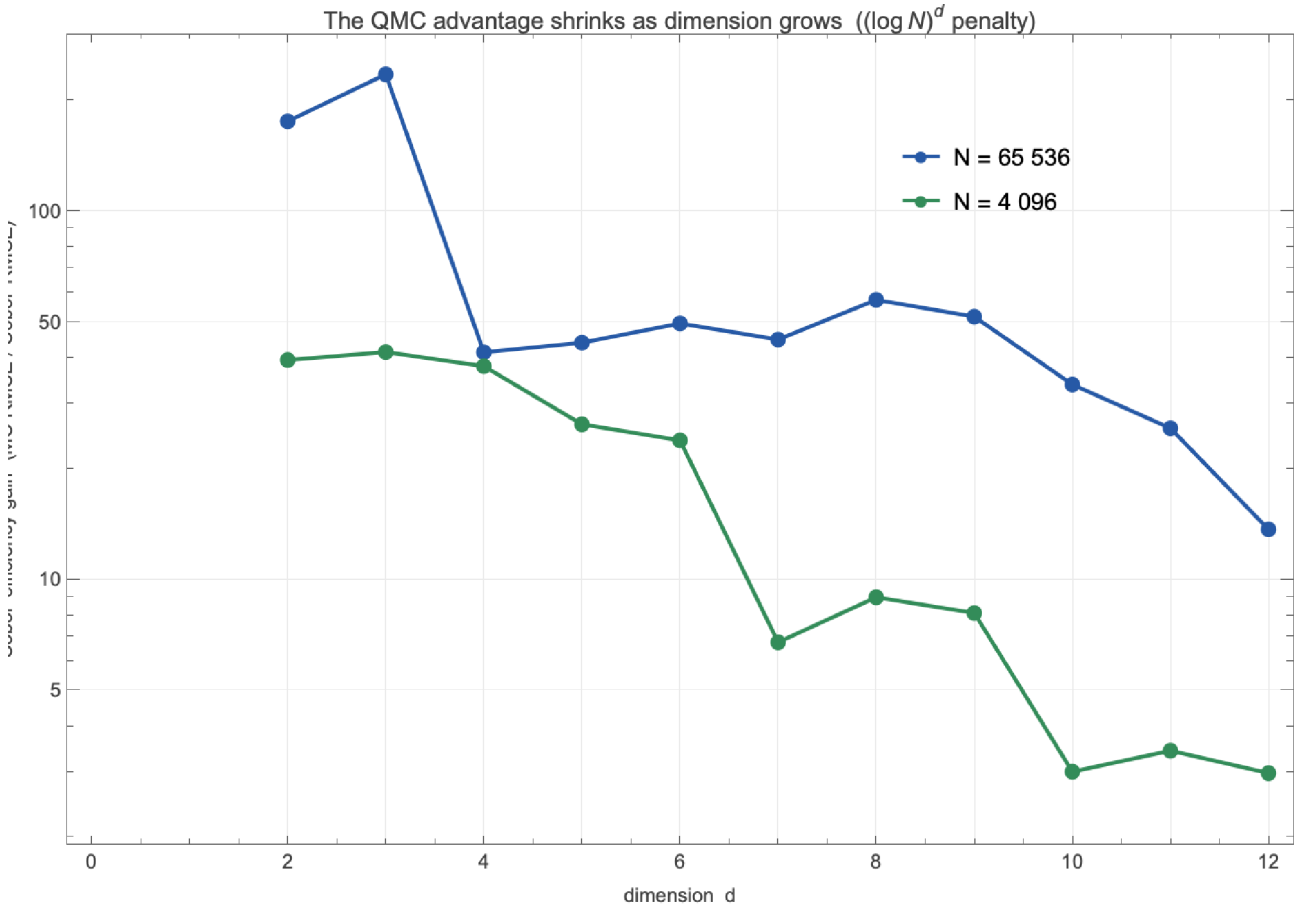

Figure 6.1 Sobol' efficiency gain (Monte Carlo RMSE divided by Sobol' RMSE, at a fixed budget) versus dimension, for two budgets. In low dimension Sobol' is tens to hundreds of times more accurate per sample; the gain decays steadily as d climbs and the $(\log N)^d$ penalty bites, heading toward parity (grey line). The six-parameter PK problem of this post sits where the gain is still large — the QMC sweet spot.

The practical lesson: QMC is most valuable in low-to-moderate dimension (say $d \le 20$), or whenever the integrand has low *effective* dimension — a few directions that matter and many that barely do. Many Bayesian models, including this one, are exactly that.

7. Correctness: synthetic-truth recovery

A fast wrong answer is worthless, so we check that the QMC machinery recovers parameters we *know*. We generate concentrations from a fixed "true" parameter vector plus noise, fit the same model, and ask whether the posterior brackets the truth:

```
(* Generate synthetic data from known truth, then fit. *)
thetaTrue = {1.20, 2.50, 30.0, 1.80, 22.0, 0.45};
{synProblem, synConc} = SyntheticProblem[thetaTrue, 320.0,
   {0.25, 0.5, 1, 1.5, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24, 30},
   priorMedians, priorLogSD, 4242];
synDraws = PosteriorDraws[LDPoints["Sobol", 8192, 6],
   LaplacePosterior[synProblem]];
Mean /@ Transpose[synDraws]                    (* posterior means *)
```

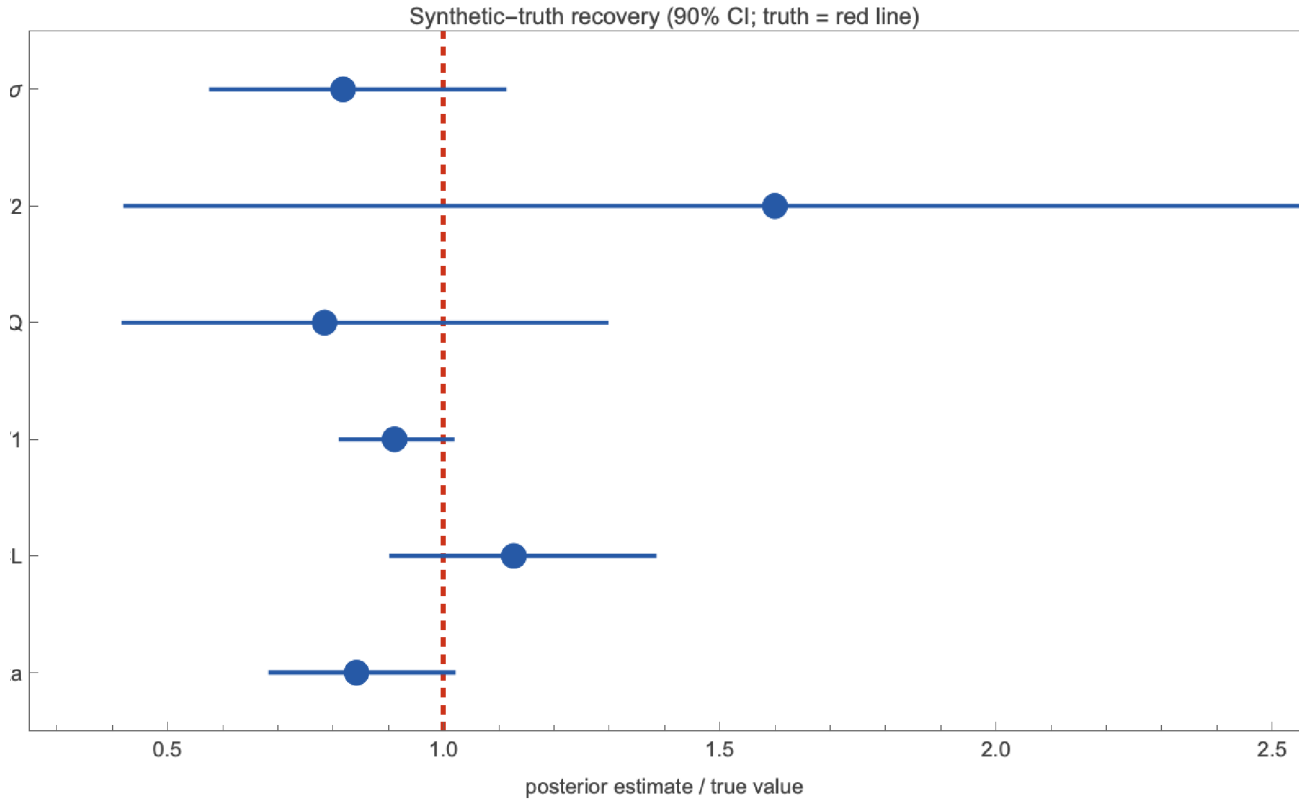

Figure 7.1 Synthetic-truth recovery. Each parameter is shown as its posterior estimate divided by the true value, with a 90% credible interval; the red line at 1.0 is the truth. The well-identified parameters (k_a, CL, V_1) land tight on the truth; the weakly-identified Q and V_2 have wide intervals that still cover it. The method is correct, and it reports honest uncertainty where the data are uninformative.

8. The practical pay-off

Slopes are abstract; model solves are money. The question a practitioner asks is: *to reach the accuracy I need, how many times must I run the forward model?* Inverting the fitted error laws — $N \propto \text{tol}^{-2}$ for Monte Carlo, $N \propto \text{tol}^{-1}$ for QMC — gives the count directly, and the ratio is the speed-up:

```
(* Forward-model solves to reach a target accuracy on E[C(6 h)]. *)
fitPow[e_] := Module[{c = Fit[Transpose[{Log[Nlist], Log[e]}], {1, x}, x]},
   {Exp[c /. x -> 0], Coefficient[c, x]}];
nForTol[e_, tol_] := With[{cp = fitPow[e]}, (cp[[1]]/tol)^(1/(-cp[[2]]))];
(* speed-up of Sobol' over MC at 0.1% relative accuracy *)
nForTol[rmsePred["MonteCarlo"], 0.001 refPred] /
   nForTol[rmsePred["Sobol"], 0.001 refPred]
```

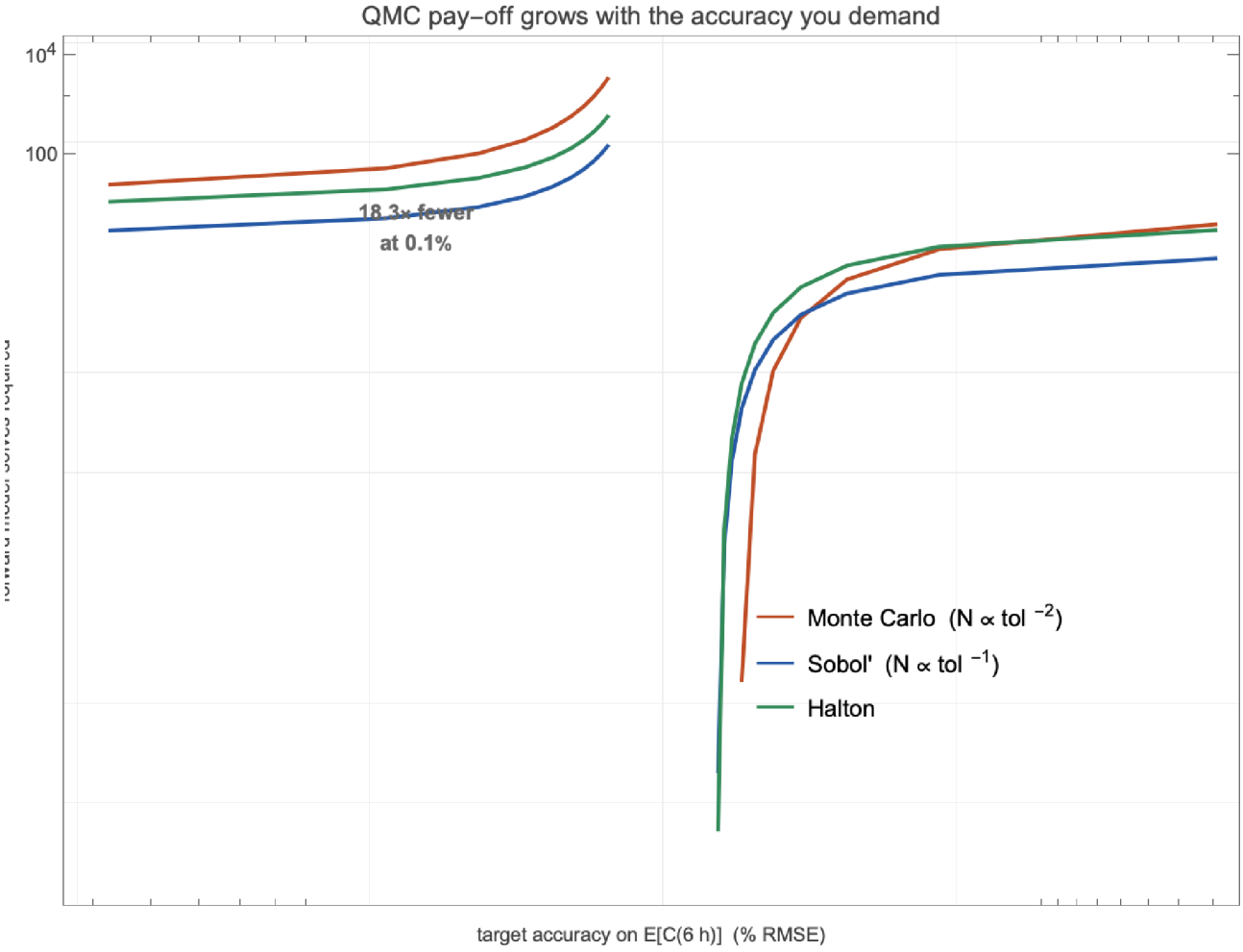

Figure 8.1 Forward-model solves required to reach a given accuracy on the posterior-predictive concentration. Because Monte Carlo needs $O(tol^{-2})$ evaluations while QMC needs $O(tol^{-1})$, the gap widens as you demand more accuracy: Sobol' is about 2× cheaper at 1% accuracy, 4× at 0.5%, and roughly 18× at 0.1%. When each evaluation is an expensive ODE solve, that is the difference between an overnight run and a coffee break.

This is the whole value proposition in one figure. QMC does not change what you compute — same posterior, same estimand, same code structure — it changes only *which points you evaluate at*, and that single swap buys back most of an order of magnitude (often more) in model runs.

9. Conclusions and caveats

On a genuine six-parameter pharmacokinetic inference from the Theophylline data, deterministic low-discrepancy sampling beats plain Monte Carlo by close to a full power of N: measured slopes of ≈ -0.95 for Sobol' and Halton against ≈ -0.5 for MC, and an order-of-magnitude saving in forward-model solves at the accuracies that matter.

When QMC helps, and when it does not. The gains are real when (i) the dimension is low-to-moderate or the effective dimension is small; (ii) the integrand is smooth, i.e. of bounded variation; and (iii) evaluations are the bottleneck. They evaporate when the integrand is rough or discontinuous (indicator functions, hard thresholds), when the dimension is very large, or when an importance-sampling weight is heavy-tailed — which is exactly why we integrate against a smooth Gaussianized posterior rather than a spiky importance ratio.

Reproducing the rate in your own work. The recipe is portable: (1) reparameterize to an unconstrained space and build a Gaussian (Laplace) approximation of the posterior; (2) map the unit cube through it with the inverse-CDF transform; (3) replace your pseudo-random draws with a randomized Sobol' or Halton set; (4) average over a few randomizations to get an error bar. Steps 1–2 are standard Bayesian asymptotics; steps 3–4 are a two-line change to existing code.

10. References

Halton, J. H. (1960). On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik* 2, 84–90.

Sobol', I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Comp. Math. and Math. Phys.* 7, 86–112.

Joe, S., & Kuo, F. Y. (2008). Constructing Sobol' sequences with better two-dimensional projections. *SIAM J. Scientific Computing* 30, 2635–2654. [The direction numbers used here.]

Niederreiter, H. (1992). *Random Number Generation and Quasi-Monte Carlo Methods*. SIAM. [Koksma–Hlawka, discrepancy theory.]

Caflisch, R. E. (1998). Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica* 7, 1–49. [Effective dimension; when QMC wins.]

Owen, A. B. (1998). Scrambling Sobol' and Niederreiter–Xing points. *J. Complexity* 14, 466–489. [Randomized QMC and error estimation.]

Gerber, M., & Chopin, N. (2015). Sequential quasi-Monte Carlo. *J. Royal Statistical Society B* 77, 509–579. [QMC inside Bayesian computation.]

Boeckmann, A. J., Sheiner, L. B., & Beal, S. L. (1994). *NONMEM Users Guide*. University of California, San Francisco. [Source of the Theophylline data set.]

Gabrielsson, J., & Weiner, D. (2016). *Pharmacokinetic and Pharmacodynamic Data Analysis* (5th ed.). [Two-compartment oral model.]

11. Reproducibility

All source code lives in the public repository **github.com/mthiel74/QuasiMonteCarloBayesianEstimation** (`https://github.com/mthiel74/QuasiMonteCarloBayesianEstimation`). The pipeline is pure Wolfram Language. To rebuild from scratch with Wolfram Language 14+:

```
# 1. Fetch the Theophylline data into data/
wolframscript -file wolfram/fetch_theoph.wls

# 2. Render every figure into docs/images/
wolframscript -file wolfram/run_all.wls

# 3. Rebuild this notebook
wolframscript -file community/build_notebook.wls
```

The companion package `community/qmc_bayes_helpers.wl` is the single source of truth for the generators, the model, and the estimators; the figure scripts and this notebook both load it. Every code cell above calls into it with all arguments shown inline, so the post doubles as runnable documentation.

Generated on June 3, 2026.